

Symbolic Relational Deep Reinforcement Learning based on Graph Neural Networks

Jaromír Janisch¹ Tomáš Pevný¹ Viliam Lisý¹

Abstract

We focus on reinforcement learning (RL) in relational problems that are naturally defined in terms of objects, their relations, and manipulations. These problems are characterized by variable state and action spaces, and finding a fixed-length representation, required by most existing RL methods, is difficult, if not impossible. We present a deep RL framework based on graph neural networks and auto-regressive policy decomposition that naturally works with these problems and is completely domain-independent. We demonstrate the framework in three very distinct domains and we report the method’s competitive performance and impressive zero-shot generalization over different problem sizes. In goal-oriented BlockWorld, we demonstrate multi-parameter actions with pre-conditions. In SysAdmin, we show how to select multiple objects simultaneously. In the classical planning domain of Sokoban, the method trained exclusively on 10×10 problems with three boxes solves 89% of 15×15 problems with five boxes.

1. Introduction

In this work, we take a fresh look at Relational Reinforcement Learning (RRL; Džeroski et al., 2001) with the modern approaches of Deep Reinforcement Learning (Deep RL). In contrast to the modern trend of applying Deep RL on the raw visual input (e.g., Mnih et al., 2015; Jaderberg et al., 2019), we target tasks of relational and symbolic nature. The world in these tasks consists of discrete *objects*, their *relations*, and *actions* that directly manipulate them. Such tasks naturally occur all around us. For example, a sim-

¹Artificial Intelligence Center, Department of Computer Science, Faculty of Electrical Engineering, Czech Technical University in Prague. Correspondence to: Jaromír Janisch <jaromir.janisch@fel.cvut.cz>.

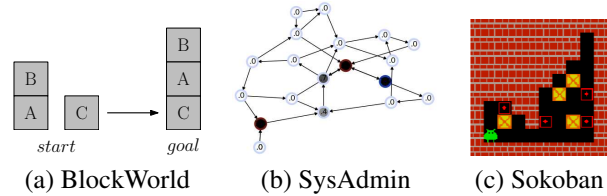


Figure 1: Three domains where we demonstrate our method. In the BlockWorld game, the task is to reconfigure blocks into a goal position. In SysAdmin, computers in a network where nodes need to be selectively restarted to keep them running. Sokoban is a classic planning domain, where we include a twist – actions are applied directly on the objects. In all three domains, **the key challenge is to zero-shot generalize to different problem sizes.**

ple task of cooking integrates a complex system of objects in various relations and possible actions. Online services with an API, social networks, computer network penetration testing, medical diagnosis, and factory assembly line optimizations are examples of these environments.

To illustrate the setting, look at BlockWorld (Slaney & Thiébaux, 2001) in Figure 1a. Initially, several labeled blocks are stacked on top of each other in an arbitrary configuration. The task is to reconfigure them into a goal position, using a *move* action that picks a block and puts it on top of another, or to the ground. The challenge is to approach the problem in a truly symbolic way. That is, using the *move* action with two parameters, *without being fixed to a pre-defined set of blocks*. Current Deep RL focuses mostly on visual-control domains (e.g., Leibo et al., 2018) that differ mainly in their fixed state and action space dimensions. Some of the described problems could be transformed into visual control domains. However, for many tasks, a natural and easier representation is within the relational framework.

In planning, Relational Dynamic Influence Diagram Language (RDDL; Sanner, 2010) is often used to describe the mechanics of a relational domain. Recently, Garg et al. (2020) introduced SymNet, a method that automatically extracts objects, interactions, and action templates from any RDDL. The interacting objects are joined to tuples and represented as a node in a graph; the interactions are represented

as edges. Graph Neural Network (GNN; Zhou et al., 2018) is used to create nodes’ embeddings. Actions templates, each represented as a single non-linear function, are applied over the object tuples to create a probability distribution. Finally, actions and their parameters (the object tuples) are selected, and the model is updated with a policy gradient method (Li, 2018).

There are several drawbacks to the SymNet algorithm. Although the actions share their representation, the final step is to apply the softmax function over all grounded actions. For a problem with n objects and an action with p parameters, this results in $O(n^p)$ time and space complexity. For actions where any subset of objects can become a parameter (*set actions*), it is $O(2^n)$. Also, the method is applicable only when the RDDDL domain definition is available because it uses the defined transition dynamics to create the graph. In Deep RL, it is common that the transition dynamics are unknown, and only a simulator is available. For example, imagine a visual control domain (e.g., controlling a robotic hand) with automatic object detection (e.g., Redmon & Farhadi, 2018), defined actions manipulating these objects (possibly with a low-level planner in place), and unknown dynamics.

This work presents a Symbolic Relational Deep RL (SR-DRL) framework that addresses these issues. It accepts an enriched symbolic input (i.e., objects and their relations, optionally augmented with their features) in the form of a graph, actions definitions and does not require the knowledge of transition dynamics. The framework is designed to generalize well over an arbitrary number of objects. Its main components are based on graph neural networks (Zhou et al., 2018), auto-regressive policy decomposition (Vinyals et al., 2017), and policy gradient methods (Li, 2018). Compared to SymNet, our framework works within $O(pkn)$ time and $O(n)$ space complexity for multi-parameter actions (where k is a maximal node degree in the graph and is usually small) and $O(n)$ time and space complexity for set actions.

We showcase the framework in three domains. BlockWorld is a well-known planning domain with NP-hard complexity for optimal planning. Here, we demonstrate a multi-parameter action with a conditional dependency. Sokoban is a game requiring extensive planning. We show how to manipulate the game’s objects on the macro level, with a low-level planner behind. SysAdmin is a graph-based planning domain, where we demonstrate our framework’s capability to select multiple nodes at once. In all three domains, we show an impressive zero-shot generalization to different problem sizes. In BlockWorld, the agent trained only on five blocks can solve problems with 20 blocks with a 78% success rate. In Sokoban, the agent trained in 10×10 problems with three boxes solves 89% of random 15×15 problems with five boxes. In SysAdmin, the agent trained with 10 nodes generalizes almost perfectly to 160 nodes.

1.1. Related work

Payani & Fekri (2020) replace the hard logic of Džeroski et al. (2001) with a differentiable inductive logic programming. In their approach, the logic predicates are fuzzy, and the parameters are learned with gradient descent. BlockWorld environment is also used for experiments. However, the scope is very limited to only 4 and 5 blocks, without goal generalization (the goal is to always stack into a single tower), and the authors do not report any generalization.

Li et al. (2019) studies a 3-dimensional instantiation of the BlockWorld problem with a robotic hand and physics simulation. Interestingly, the features of the blocks (their position and color) and the hand are encoded as a graph and processed by a graph neural network (GNN) (Zhou et al., 2018), making it invariant to the number of objects. Still, all interaction with the world is done by controlling the robotic hand and its elementary actions (relative change of position and grasping controls). Moreover, the blocks are not symbolic, but are identified by their features (color).

Hamrick et al. (2018) study the stability of a tower of blocks in a physical simulation, with some blocks glued together. The blocks and their physical features are encoded as nodes in a graph, and the actions are performed on the graph’s edges, which connect two adjoining blocks. The actions are performed onto the blocks themselves, making this approach generalize well to different combinations and numbers of blocks. Similarly, Bapst et al. (2019) focus on a task of creating block structures under physical simulation. They follow a similar approach to ours; their architecture is based on GNN and allows object-centric actions. However, these works focus on their specific domain and do not provide a general framework. Comparatively, we describe a domain-independent framework that works with heterogeneous relations, and multi-parameter and set actions.

Zambaldi et al. (2019) and Santoro et al. (2017) provide specialized neural network architectures with relational inductive biases that internally segment a visual input into objects and process them relationally. Compared to our work, these architectures cannot process symbolic input.

Garg et al. (2019) describe a GNN-based model trained with Deep RL to learn an RDDDL specified domain with zero-shot generalization to different domain variations. However, it is restricted only to single-parameter actions. In subsequent work, Garg et al. (2020) remove this restriction and present a different approach that automatically processes any RDDDL to create a domain-specific GNN-based architecture. We discuss the main differences in the Introduction.

Adjodah et al. (2018) studies a control problem of maze-navigation with a relational module. However, the studied problem is heavily restricted. It is a fixed size grid, and the relations are represented as exhaustive binary combinations

of all places in the grid. A relational module’s output is encoded as a fixed-size embedding and processed with a standard MLP based Q-learning algorithm. No message passing is involved, allowing the model to reason only with the limited binary relations.

In visual domains where a relational description is not available, work of Garnelo et al. (2016) or Zelinka et al. (2019) could be used for automatic object discovery.

2. Problem

Our problems naturally consist of *objects*, binary *relations*, *global context*, and a *goal* definition. Our problems are sequential; hence we assume existing *transition* dynamics. We use the Markov Decision Process (MDP) formalism for the following definitions. The MDP is a tuple $(\mathcal{S}, \mathcal{A}, r, t, \gamma)$, where $\mathcal{S}$, $\mathcal{A}$ represent the state and action spaces, r , t are reward and transition functions, and γ is the discount factor.

All of the MDP components are problem-dependent. The reward function r can directly specify the goal or be more subtle. For example, in the BlockWorld, the reward can be defined as a small negative value (e.g., -0.1) per step and a non-negative value when the goal is reached. The parameter γ and the transition function t are directly defined by the particular environment. Because we use a model-free method, the transition function can also be unknown (only a simulator is needed). The definition of state and action spaces is more complex and is described below.

2.1. State encoding

A state includes the key components – objects and their features, relations, global context, and optionally the goal. The objects and relations naturally form an oriented graph. Nodes represent the objects and contain their features in the form of a fixed-length vector. More complicated feature structures can be embedded using the existing techniques (e.g., Pevný & Somol, 2016; Zaheer et al., 2017). Heterogeneous objects can be recognized by a type-specifying feature. Oriented edges represent the relations, including their features and type. Symmetric relations can be transformed into two opposite edges. The global context is a vector specifying properties of the environment, unrelated to any single object (e.g., time or the environment state).

In domains where the goal is static, it can be encoded using the reward function. For instance-wise goals, it needs to be encoded in the state. Depending on the particular domain, the goal can be encoded either in the global context, in the object features, or as part of the graph itself.

The particular definition of the state space $\mathcal{S}$ is problem-dependent. As an example, Figure 2 illustrates a possible state encoding of the BlockWorld game state previously

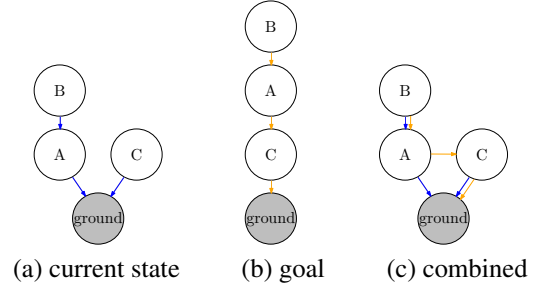


Figure 2: Example state encoding of the BlockWorld game state from Figure 1a. The objects and relations form the graph, with a special node representing the ground. The representation of the current state and the goal is combined with different edge types. Nodes include a single feature differentiating the blocks and ground; no other features (e.g., labels) are present. Likewise, no global information is needed in this example.

shown in Figure 1a. Objects and relations are encoded as a graph, with a special node representing the ground (see BlockWorld definition for details). Nodes contain only a single feature, differentiating between a regular node and the ground. The actual state and the goal are encoded with different edge types, and their representation is combined.

2.2. Actions

The problems of our focus require actions with symbolic parameters, i.e., the objects. An action is composed of its label, set of parameters, and pre-conditions. We call actions without any parameters *elementary* (e.g., *turn-left*). Parameters of actions can be conditionally dependent or not. For example, in an action $move(x, y)$ with two parameters x, y , the choice of parameter y depends on the chosen x . As a special action class, we introduce *set* actions, where any subset of objects can become a parameter (e.g., $select(\mathcal{X})$, where $\mathcal{X}$ is an arbitrary set of nodes). An action is available only if all its pre-conditions in a particular state are met. As an example, an $open(x)$ action may be unavailable unless the object x contains a feature specifying it is closed.

The action space $\mathcal{A}$ is problem-dependent and also state-dependent (the number of objects may change, pre-conditions may disable some actions). For instance, in the game of BlockWorld, there is a single two-parameter action $move(x, y)$ with a pre-condition that there is no block on top of x , and neither on y (unless y is the ground).

3. Method

Our method uses GNNs (Zhou et al., 2018) to process the complex state. To tackle the multi-parameter actions, we use auto-regressive policy decomposition (Vinyals et al.,

2017). Our model is fully differentiable and can be trained by any policy gradient algorithm (Li, 2018).

3.1. Graph Neural Network

Because the state is represented as a graph, the natural choice is to use GNNs, which have strong relational inductive biases (Battaglia et al., 2018) and their operations are local and invariant to node permutations. Also, the same model can be used to process states with a different number of objects. Several GNN variations exist, with a unifying framework made by Battaglia et al. (2018). We use a custom implementation that includes node and edge features, skip connections, a global node with an attention mechanism, and separate parameters for each message passing step. The exact details are given in the Appendix.

The GNN accepts the state graph $(\mathcal{V}, g, \mathcal{E})$, where $\mathcal{V}$ are nodes, $\mathcal{E}$ are oriented edges, and g is a special *global* node. If available, g can initially contain the global context. Several message-passing steps are performed, and the final embeddings are saved in $\mathcal{V}$ and g . For simplicity, let $v \in \mathcal{V}$ and g also denote the final embedding vector of the respective nodes.

3.2. Policy decomposition

The policy $\pi(s)$ is a probability distribution over all possible actions in a state s . Usually, the action space grows exponentially with the number of actions' parameters. However, such policy can be decomposed into a sequence of choices and the complexity lowered to linear. For now, let a be a particular choice of action with its parameters, denoted as a tuple $a = (a_0, a_1, \dots, a_{L(a)})$. Here, a_0 denotes the action identifier (e.g., *stop*, *move*, etc.), $a_1, a_2, \dots$ its parameters, and $L(a)$ its arity. The action parameters are the graph's nodes, $a_{1..|L(a)|} \in \mathcal{V}$. The policy can then be represented in an auto-regressive manner (Vinyals et al., 2017):

$$\pi(a|s) = \pi_0(a_0|s) \prod_{l=1}^{L(a)} \pi_{a_0,l}(a_l|a_{<l}, s)$$

where π_0 is the policy selecting the action identifier, and $\pi_{a_0,l}$ is the policy selecting a parameter l of the action a_0 .

In their method, Vinyals et al. (2017) chose to disregard the conditional dependency on the previously chosen parameters. However, this variant cannot represent every possible probability distribution, and for some actions, the previously selected parameters are crucial for further selection. E.g., in *move*(x, y), the selection of y only makes sense with a known x . Therefore, we propose a method to preserve the conditional dependency below (see Figure 3).

In the previous step, the GNN processed the state information into node embeddings $\mathcal{V}$ and the global context g ;

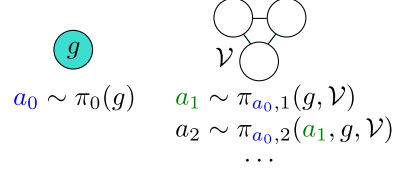


Figure 3: The embeddings of nodes $\mathcal{V}$ and the global context g created by the GNN are used to select actions. First, the action identifier a_0 (e.g., *move*) is selected, based on the global context g . Then, parameters $a_1, a_2, \dots$ are sequentially selected, conditioned on the previous selection.

hence $\pi(s) = \pi(g, \mathcal{V})$. If we disregard the pre-conditions for now, the number of a_0 actions is static. Therefore, the action identifier can be selected only with the global context, $a_0 \sim \pi_0(g)$. We implement π_0 as a single softmax layer, with one output for each a_0 .

For elementary actions without parameters, the decision ends here. Otherwise, the parameters are dependent on the action identifier and previously selected parameters, $a_l \sim \pi_{a_0,l}(a_1, \dots, a_{l-1}, g, \mathcal{V})$. The first parameter can be chosen directly, $a_1 \sim \pi_{a_0,1}(g, \mathcal{V})$. To select parameters when $l \geq 2$, the information about previous selections has to be given to the model. We implement this by augmenting the existing node embeddings with one-hot encoding of previous selection. Every node $v \in \mathcal{V}$ is augmented with a binary vector z^v of size $l-1$, where z_i^v denotes whether the node v was selected as a i -th parameter, $z_i^v = 1$ if $a_i = v$, else 0. To preserve the original embedding size, the augmented vector is transformed to its original size with a single non-linear layer. Next, two message-passing steps are used to allow the information to spread, resulting in new embeddings $\mathcal{V}', g'$. The choice is then made, $a_l \sim \pi_{a_0,l}(g', \mathcal{V}')$. Finally, the embeddings $\mathcal{V}', g'$ are discarded, and the process repeats for the next parameter with original $\mathcal{V}, g$. Each policy $\pi_{a_0,i}$ is implemented as a linear layer with shared parameters applied to each node v and taking softmax.

Pre-conditions determine whether an action is available in a particular situation. The availability is resolved for the currently processed level (e.g., for $a_0, a_1, \dots$), and the unavailable actions are removed from the softmax computation. In the most general case, no selection may be possible for a particular level l . In that case, the algorithm has to backtrack, disable the selection at level $l-1$ that led to the situation and select a new parameter.

Set actions accept an arbitrary subset of nodes as their parameter. To perform such selection, we use concurrent actions (Harmer et al., 2018). A shared function with sigmoid activation is used to compute per-node probabilities $p(v)$. Then, nodes are selected with independent Bernoulli trials. Let Υ be the set of selected nodes; the total probability for this action is then $\pi_0(a_0|g) \prod_{v \in \Upsilon} p(v) \prod_{v \in \mathcal{V} \setminus \Upsilon} (1 - p(v))$.

Complexity analysis. In a graph with n nodes and a maximal degree k , the time complexity of a message passing step is $O(kn)$. Parameters of an action with p parameters are selected sequentially, and two message passes are performed for each. The time complexity of selecting the action is then $O(pkn)$, while k is usually very small. In the case of set actions, it is $O(n)$, as each node is selected independently. Information can be sequentially accumulated in nodes; hence space complexity is $O(n)$ for both parametrized and set actions.

3.3. Model training

Let θ be the model parameters – a union for all ϕ functions and layers used in the action selection. Apart from the parametrized policy π_θ , the model outputs a value estimate of a state, dependent on the final global context, $V_\theta(s) = V_\theta(g)$, implemented as a single linear layer. Although the action selection involves deterministic choices of their parameters, the final product $\pi_\theta(a|s)$ is fully differentiable. We propose to use A2C algorithm, a synchronous version of A3C (Mnih et al., 2016), with a few modifications. More details about the implementation are in the Appendix.

4. Experiments

To demonstrate our method’s generality and performance, we provide an implementation¹ and experimental results in three distinct domains. Each of the tested domain focus on a slightly different aspect. In BlockWorld, we showcase a two-parameter action with conditional dependency between the parameters and pre-conditions. In Sokoban, five single-parameter actions are used. The domain is modified to show that the framework can issue macro-actions that control the environment on the level of objects, while a low-level planner breaks them to micro-actions. SysAdmin is a graph domain where we demonstrate set actions.

We provide only a brief description of each domain and how the state and actions are modeled due to the limited space. For precise definitions, please see the Appendix, which also describes the model architecture and hyper-parameters.

Time-limits. The used environments are not restricted by any time horizon, hence we use a discount factor $\gamma = 0.99$ in all domains. To enhance the training experience diversity and avoid possible deadlocks (e.g., in Sokoban), we employ an artificial step limit (100 in BlockWorld and SysAdmin, 200 in Sokoban). This limit is regarded as auxiliary and not part of the environment, in the spirit of Pardo et al. (2018).

Reference machine. When we report our algorithm’s running times in the following text, we are using our reference machine equipped with AMD Ryzen 1900X CPU, 8 GB of

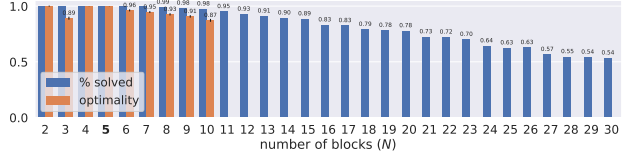


Figure 4: The agent trained in BlockWorld with 5 blocks ($N = 5$) is evaluated in problems with $N \in \{2..30\}$. For each N , the agent is evaluated in 1000 problems, and the percentage of solved problems and its optimality is reported.

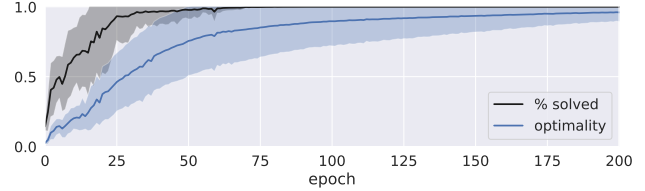


Figure 5: Training in the BlockWorld environment, $N = 5$, each epoch equals 256k environment steps. The graph shows the percentage of solved problems and optimality (optimal / performed steps). The mean $\pm$ one standard deviation of eight randomly initialized runs is displayed.

RAM, and nVidia Titan X GPU.

4.1. BlockWorld

BlockWorld is a well-known domain with tractable satisficing planning and NP-hard optimal planning (Slaney & Thiébaux, 2001). This environment consists of N blocks and a special ground object. The blocks can be placed on top of each other or on the ground. A single $move(x, y)$ action with two parameters is available, which picks a block x and put it on top of y . Its pre-conditions are that x and y are free, unless y is the ground. The goal is to reconfigure the blocks from a starting position to a given goal position. The agent receives a small penalty per each step and a reward for solving the problem. See Appendix for the exact definition.

We trained eight models with different seeds in the BlockWorld environment with $N = 5$ and randomly generated initial states and goals. The agent is evaluated on 1000 random problems with $N = 5$, and we report the percentage of solved problems and optimality – the average ratio of the number of optimal steps and performed steps for each problem. In case the agent does not solve the environment in the 100 step limit, we consider the ratio to be 0. Figure 5 shows the first 200 epochs, where a single epoch is 256k environment steps. On our reference machine, a single epoch takes about 3.3 minutes; 100 epochs about 5.5 hours.

At about epoch 75, the agent learns to solve the problems with 100% accuracy and 83% optimality. Subsequently, the optimality increases; in epoch 200 it’s 96%, and it reaches

¹The code is at github.com/jaromiru/sr-drl.

99% in epoch 400. Hence, it can be said that the agent is able to learn near-optimal policy in this setting.

In the next experiment, we investigated whether the agent can solve this environment without pre-conditions. That is, all actions are available, but the nonsensical actions do not change the state. Our experiment confirms that the agent learns to ignore the nonsensical actions; however, the training time is almost doubled. In this case, the agent solves 100% of problems at about epoch 130 and reaches 99% optimality at about epoch 730. We conclude that the pre-conditions are not necessary, but greatly help the training.

4.1.1. AGENT’S GENERALIZATION

Next, we focused on the agent’s generalization to a different number of blocks. From the eight runs with $N = 5$, we picked the one that performed best after 800 epochs. Next, we evaluated it in environments with a different number of blocks, $N \in \{2..30\}$. Again, we measured the percentage of solved environments (within the 100 step limit) and the agent’s optimality. Because in BlockWorld, the optimal planning is NP-hard, we report the optimality only for $N \leq 10$; it becomes too expensive to compute with higher N . The results, reported in Figure 4, show impressive generalization capabilities. The agent zero-shot generalizes with great success to other problem sizes. It solves all problems for $N \leq 5$ with near-100% optimality, with an exception of $N = 3$. With $N \geq 6$, the fraction of solved problems gradually decreases to 78% for $N = 20$ and 54% for $N = 30$. The optimality decreases to 87% for $N = 10$.

When we tried to train an agent directly with $N = 10$, the training failed. Yet, the agent trained with $N = 5$ is able to solve 98% of problems with $N = 10$, with 87% optimality. Moreover, it gracefully generalizes up to $N \leq 30$, possibly even more. This indicates a strong potential for curriculum learning (Bengio et al., 2009). To understand how impressive these results are, we note that the number of all possible block configurations raise very quickly with N . Exactly, it is $\sum_{i=0}^N \binom{N}{i} \frac{(N-1)!}{(i-1)!}$ (Slaney & Thiébaux, 2001); i.e., 501 for $N = 5$, 5.8×10^7 for $N = 10$ and 2.7×10^{20} for $N = 20$. The number of actions is $|\mathcal{A}| \leq N^2$.

4.1.2. PRIOR-ART

Džeroski et al. (2001) investigated RRL with inductive logic programming. They focused on a more restrictive BlockWorld variant, with only three specific goals: stacking into a single tower, unstacking everything to the ground, or moving a specific box a on top of b . For each of these goals, a specialized policy was created, and the authors also reported some degree of generalization to a different number of blocks (3 to 10). However, the used specific goals are trivial compared to the setting we use (any block configura-

tion as a goal). Also, due to a different evaluation procedure, the exact comparison is impossible without carefully reimplementing and evaluating their method.

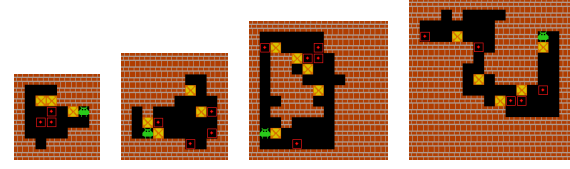
Recently, Payani & Fekri (2020) approached the RRL problem with differentiable inductive logic programming, in which a set of predicates is defined, and their probabilities are learned by gradient descent. The advantage of the approach is that alternative high-level predicates can be included to facilitate the learning. The work also focuses on the BlockWorld problem, with an input represented as an image. However, it is unclear if their approach scales and generalizes, as the authors only train and evaluate their algorithm with 4 and 5 blocks.

4.2. Sokoban

Sokoban (see Figure 6) is a classic planning domain, where an agent moves inside a grid maze with the goal of pushing boxes onto their destination. Solving levels requires careful planning because some actions are irreversible and can lead to an unsolvable situation. Usually, the actions control the player avatar and are elementary – *left*, *up*, *right*, and *down*. To demonstrate our framework’s power, we decided to define new **actions that operate directly on the boxes**, with a low-level planner that translates them into the elementary actions while preserving all the environment’s mechanics. The new actions are *push-left*(x), *push-right*(x), *push-up*(x), and *push-down*(x), and all operate on a box x , moving the player such that the box is pushed to the indicated direction, if possible. While it may seem unfair to other Deep RL methods that use elementary actions, we note that these macro actions are only possible in our framework and showcase its strength. Also, note that our framework can work with the elementary actions too.

We trained three models with a dataset of 10×10 problems with four boxes over the course of 17 days. Figure 7-left shows the test set performance measured during the training (elementary steps are used). We also show the performance of two other Deep RL-based architectures: I2A (Racanière et al., 2017) and DRC (Guez et al., 2019), as reported in the original papers (both were trained with the same dataset). I2A is based on convolutional neural networks (CNN) and learns an environment model used to simulate trajectories; I2A with 15 unrolls is reported. DRC is a recurrent CNN-based architecture; the DRC(3,3) version is reported.

After 10^9 elementary environment steps, SR-DRL solves 96% of test levels. In the same amount of steps, I2A reaches 90% solved levels, and DRC 99%. We hypothesize that the main advantage of DRC architecture is in its recurrence, allowing it to store intermediary calculations between steps and thus be much more effective. Recurrent architecture can be also used with our method and is a promising future direction. Although DRC outperforms our method in this



8x8 3 boxes	10x10 4 boxes	13x13 5 boxes	15x15 5 boxes
96.7%	96%	86.9%	89%

Figure 6: We evaluated an agent trained solely on the 10x10 levels with 4 boxes on randomly generated levels in other game variations and measured the percentage of solved levels. The agent generalizes well to different game sizes and the box count. The results show the performance of a single agent evaluated in about 1000 problems per variation. The top row shows example levels.

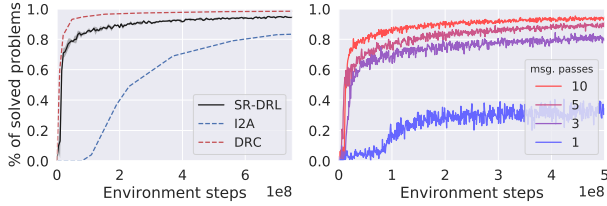


Figure 7: *Left*: Test-set performance during training on 10x10 levels with 4 boxes. SR-DRL shows mean $\pm$ one standard deviation of three models. I2A and DRC are different Deep RL based algorithms. *Right*: Number of message passes greatly influences the model’s performance. With more than 10 message passes, the model failed to train.

case, it cannot generalize to different problem sizes. Its CNN architecture ends with a fully connected layer, fixing it to the particular problem size.

Contrarily, our method is not fixed to any problem size. In Figure 6, we report results obtained by evaluating a model trained in 10x10 problems with 4 boxes on several different problem sizes. The results are impressive – the model generalizes well to both smaller and larger environments, e.g. in 8x8 with 3 boxes it solves 96.7% of problems, in 15x15 with 5 boxes it is 89%.

In a separate experiment, we tested the influence of the number of message passing steps. Figure 7-right shows the training progress with 1, 3, 5 and 10 message passes. More message passes always result in faster learning and better final performance. With more message passes (we tested 15 and 20) the model did not train at all, and the final performance was zero.

4.3. SysAdmin

SysAdmin (Guestin et al., 2003) (see Figure 8) is a probabilistic planning domain adapted from the International Probabilistic Planning Competition (IPPC) 2011. It includes stochastic transitions and an infinite time horizon, without any specific goal to reach. In this domain, the problem is defined as a graph of dependencies between N computer nodes. At each time-step, any computer can be either *on-line* or *off-line*. Online computers have a certain probability of becoming offline, based on the state of their dependencies, and offline computers have a chance to spontaneously reset and become online. We investigate two variants of the problem: In *SysAdmin-S* (*S* for *Single*), the agent can perform a *reset(x)* action, that resets a single computer. In *SysAdmin-M* (*M* for *Multi*), a set action *reset($\mathcal{X}$)* reboots an arbitrary set of computers at once. At each step, the agent is rewarded for each computer that is online at that moment and penalized for any computer it reboots.

For each $N \in \{5, 10, 20, 40, 80, 160\}$, we train eight SR-DRL agents for 50 epochs (each epoch being 25 600 environment steps). We also select a single agent trained for $N = 10$ and evaluate it in different problem variations. In each setting, we also measured the performance of PROST, an MCTS-based probabilistic planner (Keller & Eyerich, 2012; Keller & Helmert, 2013), with a time allowance of 0.1 or 10 seconds per step. We designed two baseline algorithms that we use to normalize the results: In *SysAdmin-S*, a random offline node is selected at every step (or none, if there is not any). In *SysAdmin-M*, all offline nodes are selected for a reset at each step. On our reference machine, the training time of the SR-DRL algorithm is 15/20/30/50/90/150 minutes for $N = 5/10/20/40/80/160$, respectively. During testing, the model needs about 1 ms per step.

For both *SysAdmin-S/M*, all models are evaluated in 100 problem instances with a 100 step limit. The mean results with a 95% confidence interval are reported in Figure 9. The first observation is that in all instances and both *SysAdmin-S/M*, the agent trained with $N = 10$ and evaluated for different N performs almost the same as an agent specifically trained in the respective setting. This result indicates that the policy learned for $N = 10$ is directly applicable even for $N = 160$.

When compared to PROST, the SR-DRL algorithm performs similarly in the *SysAdmin-S* variation. It performs slightly worse for $N \in \{5, 10, 20\}$. For $N = 80$ and 160, SR-DRL performs slightly better than PROST with 10s per step, while the gap widens when compared to PROST with a 0.1s step limit. We emphasize that our algorithm needs only 1ms per step to decide, after trained. I.e., for $N \geq 80$ and 100 step limit, PROST-10s needs over 16 minutes to solve the problem, while SR-DRL needs only 0.1 seconds and achieve better performance. Moreover, the training cost

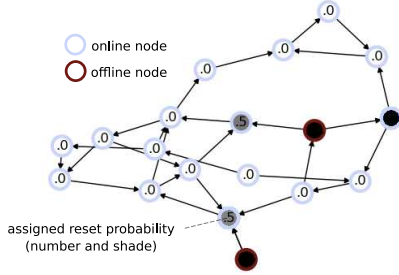


Figure 8: In SysAdmin-M, SR-DRL learns to preventively reset nodes that have a high probability of failure. The graph shows the dependency network and the reset probabilities the algorithm assigns to the nodes. Read any edge (a, b) as b depends on a .

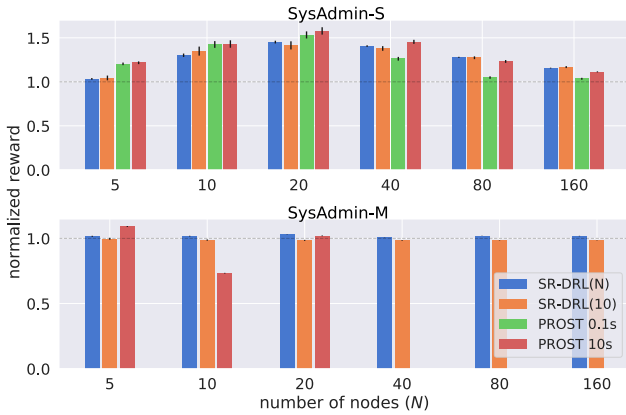


Figure 9: The figure shows normalized results in SysAdmin-S (a single node can be reset) or -M (multiple nodes can be reset at once). SR-DRL(N) is trained for the specific N , SR-DRL(10) is a model trained with $N = 10$ and evaluated in a different setting. PROST is a probabilistic planner with either 0.1s or 10s time limit per step. Rewards were normalized using baseline algorithms.

is easily amortized, especially after we showed that once trained model generalizes perfectly to different settings.

In SysAdmin-M, the first observation is that the baseline algorithm is very strong, and only PROST-10s with $N = 5$ significantly exceeds its performance. Note that with $N = 10$, PROST-10s reaches only 73% of the baseline’s performance. However strange, this result was confirmed with several repeated experiments. In this setting, PROST needs to enumerate all action combinations, resulting in $O(2^N)$ space complexity, which becomes infeasible for $N > 20$. Also, even for lower N , PROST cannot run with less than 5 seconds per step. Comparatively, the SR-DRL algorithm works well even for $N = 160$.

In Figure 8, we analyzed the behavior of an agent trained in SysAdmin-M to see whether it simply implements the

baseline algorithm (i.e., to always reset all offline nodes). We found that the algorithm learns to selectively restart even the online nodes if their failure chance is high (based on the state of their dependencies). However, it seems that the advantage over the baseline algorithm is not significant.

5. Conclusion

We presented a generic framework based on deep reinforcement learning and graph neural networks for solving relational domains. The method operates with a symbolic representation of objects, their relations, and actions manipulating them. We described a generic way to implement multi-parameter actions with mutually dependent parameters, and set actions that select an arbitrary subset of objects. Both action variations operate in linear time and space, w.r.t. the number of objects. One of the great advantages of the framework is that a trained model is not fixed to specific problem size and can be immediately applied to problems of different sizes.

We demonstrated the framework in three distinct domains, and in all, it achieved competitive performance. In addition, it showed impressive zero-shot generalization to different problem variations and sizes. In BlockWorld, the model trained solely with five objects solves 78% of problems with 20 objects, even though the state space grows exponentially with the number of objects. In Sokoban, we show that the method can be joined with a low-level planner and control the environment on its macro-level. When trained solely on 10×10 problems with four boxes, it solves 89% of 15×15 problems with five boxes. In SysAdmin, once trained model transfers almost perfectly to any other problem size. Moreover, we demonstrated the framework’s capability to select multiple objects at once with a single action. Comparatively, a widely used PROST planner cannot work in this setting with a reasonable number of objects due to its exponential complexity.

Acknowledgements

This research was supported by the European Office of Aerospace Research and Development (grant no. FA9550-18-1-7008) and by The Czech Science Foundation (grants no. 18-21409S and 18-27483Y). The GPU used for this research was donated by the NVIDIA Corporation. Computational resources were supplied by the project “e-Infrastruktura CZ” (e-INFRA LM2018140) provided within the program Projects of Large Research, Development and Innovations Infrastructures. The authors acknowledge the support of the OP VVV funded project CZ.02.1.01/0.0/0.0/16_019/0000765 “Research Center for Informatics”.

References

- Adjodah, D., Klinger, T., and Joseph, J. Symbolic relation networks for reinforcement learning. *Relational Representation Learning Workshop*, 2018.
- Bapst, V., Sanchez-Gonzalez, A., Doersch, C., Stachenfeld, K., Kohli, P., Battaglia, P., and Hamrick, J. Structured agents for physical construction. In *International Conference on Machine Learning*, pp. 464–474. PMLR, 2019.
- Battaglia, P. W., Hamrick, J. B., Bapst, V., Sanchez-Gonzalez, A., Zambaldi, V., Malinowski, M., Tacchetti, A., Raposo, D., Santoro, A., Faulkner, R., et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.
- Bengio, Y., Louradour, J., Collobert, R., and Weston, J. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pp. 41–48, 2009.
- Džeroski, S., De Raedt, L., and Driessens, K. Relational reinforcement learning. *Machine learning*, 43(1-2):7–52, 2001.
- Garg, S., Bajpai, A., and Mausam. Size independent neural transfer for RDDDL planning. In Benton, J., Lipovetzky, N., Onaindia, E., Smith, D. E., and Srivastava, S. (eds.), *Proceedings of the Twenty-Ninth International Conference on Automated Planning and Scheduling, ICAPS 2018, Berkeley, CA, USA, July 11-15, 2019*, pp. 631–636. AAAI Press, 2019. URL <https://aaai.org/ojs/index.php/ICAPS/article/view/3530>.
- Garg, S., Bajpai, A., and Mausam. Symbolic network: Generalized neural policies for relational MDPs. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pp. 3397–3407. PMLR, 2020. URL <http://proceedings.mlr.press/v119/garg20a.html>.
- Garnelo, M., Arulkumaran, K., and Shanahan, M. Towards deep symbolic reinforcement learning. *arXiv preprint arXiv:1609.05518*, 2016.
- Guestrin, C., Koller, D., Parr, R., and Venkataraman, S. Efficient solution algorithms for factored MDPs. *Journal of Artificial Intelligence Research*, 19:399–468, 2003.
- Guez, A., Mirza, M., Gregor, K., Kabra, R., Racanière, S., Weber, T., Raposo, D., Santoro, A., Orseau, L., Eccles, T., et al. An investigation of model-free planning: boxoban levels. <https://github.com/deepmind/boxoban-levels/>, 2018.
- Guez, A., Mirza, M., Gregor, K., Kabra, R., Racanière, S., Weber, T., Raposo, D., Santoro, A., Orseau, L., Eccles, T., et al. An investigation of model-free planning. In *International Conference on Machine Learning*, pp. 2464–2473. PMLR, 2019.
- Hamrick, J. B., Allen, K. R., Bapst, V., Zhu, T., McKee, K. R., Tenenbaum, J., and Battaglia, P. W. Relational inductive bias for physical construction in humans and machines. In Kalish, C., Rau, M. A., Zhu, X. J., and Rogers, T. T. (eds.), *Proceedings of the 40th Annual Meeting of the Cognitive Science Society, CogSci 2018, Madison, WI, USA, July 25-28, 2018*, 2018.
- Harmer, J., Gisslén, L., del Val, J., Holst, H., Bergdahl, J., Olsson, T., Sjöö, K., and Nordin, M. Imitation learning with concurrent actions in 3D games. In *2018 IEEE Conference on Computational Intelligence and Games (CIG)*, pp. 1–8. IEEE, 2018.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Helmert, M. The fast downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006.
- Helmert, M. and Domshlak, C. Lm-cut: Optimal planning with the landmark-cut heuristic. *Seventh international planning competition (IPC 2011), deterministic part*, pp. 103–105, 2011.
- Jaderberg, M., Czarnecki, W. M., Dunning, I., Marris, L., Lever, G., Castaneda, A. G., Beattie, C., Rabinowitz, N. C., Morcos, A. S., Ruderman, A., et al. Human-level performance in 3D multiplayer games with population-based reinforcement learning. *Science*, 364(6443):859–865, 2019.
- Keller, T. and Eyerich, P. PROST: Probabilistic planning based on UCT. In *Proceedings of the Twenty-Second International Conference on Automated Planning and Scheduling (ICAPS 2012)*, pp. 119–127. AAAI Press, 2012.
- Keller, T. and Helmert, M. Trial-based heuristic tree search for finite horizon MDPs. In *Proceedings of the Twenty-Third International Conference on Automated Planning and Scheduling (ICAPS 2013)*, pp. 135–143. AAAI Press, 2013.
- Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

- Leibo, J. Z., d’Autume, C. d. M., Zoran, D., Amos, D., Beattie, C., Anderson, K., Castañeda, A. G., Sanchez, M., Green, S., Gruslys, A., et al. Psychlab: a psychology laboratory for deep reinforcement learning agents. *arXiv preprint arXiv:1801.08116*, 2018.
- Li, R., Jabri, A., Darrell, T., and Agrawal, P. Towards practical multi-object manipulation using relational reinforcement learning. *arXiv preprint arXiv:1912.11032*, 2019.
- Li, Y. Deep reinforcement learning. *arXiv preprint arXiv:1810.06339*, 2018.
- Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. Continuous control with deep reinforcement learning. In *International Conference on Learning Representations*, 2016.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2017.
- Maas, A. L., Hannun, A. Y., and Ng, A. Y. Rectifier nonlinearities improve neural network acoustic models. In *International Conference on Learning Representations*, 2013.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533, 2015.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., and Kavukcuoglu, K. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning*, 2016.
- Pardo, F., Tavakoli, A., Levdi, V., and Kormushev, P. Time limits in reinforcement learning. In *International Conference on Machine Learning*, pp. 4045–4054, 2018.
- Payani, A. and Fekri, F. Incorporating relational background knowledge into reinforcement learning via differentiable inductive logic programming. *arXiv preprint arXiv:2003.10386*, 2020.
- Pevný, T. and Somol, P. Discriminative models for multi-instance problems with tree structure. In *Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security*, pp. 83–91. ACM, 2016.
- Racanière, S., Weber, T., Reichert, D., Buesing, L., Guez, A., Rezende, D. J., Badia, A. P., Vinyals, O., Heess, N., Li, Y., et al. Imagination-augmented agents for deep reinforcement learning. In *Advances in neural information processing systems*, pp. 5690–5701, 2017.
- Redmon, J. and Farhadi, A. YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.
- Sanner, S. Relational dynamic influence diagram language (RDDI): Language description. *Unpublished manuscript. Australian National University*, 32:27, 2010.
- Santoro, A., Raposo, D., Barrett, D. G., Malinowski, M., Pascanu, R., Battaglia, P., and Lillicrap, T. A simple neural network module for relational reasoning. In *Advances in neural information processing systems*, pp. 4967–4976, 2017.
- Schrader, M.-P. B. gym-sokoban. <https://github.com/mpSchrader/gym-sokoban>, 2018.
- Slaney, J. and Thiébaux, S. Blocks world revisited. *Artificial Intelligence*, 125(1-2):119–153, 2001.
- Vinyals, O., Ewalds, T., Bartunov, S., Georgiev, P., Vezhnevets, A. S., Yeo, M., Makhzani, A., Küttler, H., Agapiou, J., Schrittwieser, J., et al. Starcraft II: A new challenge for reinforcement learning. *arXiv preprint arXiv:1708.04782*, 2017.
- Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., and Smola, A. J. Deep sets. In *Advances in neural information processing systems*, pp. 3391–3401, 2017.
- Zambaldi, V., Raposo, D., Santoro, A., Bapst, V., Li, Y., Babuschkin, I., Tuyls, K., Reichert, D., Lillicrap, T., Lockhart, E., et al. Deep reinforcement learning with relational inductive biases. In *International Conference on Learning Representations*, 2019.
- Zelinka, M., Yuan, X., Cote, M.-A., Laroche, R., and Trischler, A. Building dynamic knowledge graphs from text-based games. *arXiv preprint arXiv:1910.09532*, 2019.
- Zhang, Y., Vuong, Q. H., Song, K., Gong, X.-Y., and Ross, K. W. Efficient entropy for policy gradient with multidimensional action space. *arXiv preprint arXiv:1806.00589*, 2018.
- Zhou, J., Cui, G., Zhang, Z., Yang, C., Liu, Z., and Sun, M. Graph neural networks: A review of methods and applications. *arXiv preprint arXiv:1812.08434*, 2018.

A. Domain definitions

A.1. BoxWorld

Definition. The objects in the BlockWorld problem consist of a set of N blocks $\mathcal{B} = \{b_1, b_2, \dots, b_N\}$ and a special object G , representing the ground. Let's define a relation $x \dashv y; x \in \mathcal{B}, y \in \mathcal{B} \cup G$, meaning that a block x is positioned on top of y . For each x , the relation is unique, as well as for each y , unless $y = G$. Let $\mathcal{R}$ be a set of all relations in the problem. The action $move(x, y)$ removes all relations $x \dashv z; \forall z$ from $\mathcal{R}$ and creates a new one $x \dashv y$. The pre-conditions for the action are $x \neq y$, $free(x)$ and $free(y) \vee y = G$, where $free(x) \Leftrightarrow \nexists z : z \dashv x$.

The goal is to use the action $move$ to reconfigure the block positions $\mathcal{R}_{start}$ into $\mathcal{R}_{goal}$. To incentivize the agent to find the optimal solution, it receives a reward -0.1 for each action. After reaching the goal, the episode ends with a reward 10.

State and actions. The state consists of the objects $\mathcal{B}, G$, the current set of relations $\mathcal{R}$, and the goal $\mathcal{R}_{goal}$ (see Figure 2 for illustration). In the graph, each relation is modeled symmetrically (both *above-of* and *below-of* are included). The different types of relations are marked by their edge parameters. The objects contain a single-bit feature that signifies whether they belong to $\mathcal{B}$ or G . Note that no block labels are present in the state in any way.

There is a single action $move$ with two parameters. The pre-conditions are used according to their definition. If a particular block is allowed to be the first parameter of the action $move$, there always exists a valid second parameter (e.g., G).

Generation. A problem is generated as follows. From a set of N available blocks $\mathcal{B}' = b_1, \dots, b_N$ a random subset of 1 to $|\mathcal{B}'|$ blocks is chosen and stacked in random order. This stack is then removed from $\mathcal{B}'$, and the procedure repeats until $\mathcal{B}'$ is empty. The goal is generated in the same way.

A.2. Sokoban

Definition. We use the Sokoban environment as defined in Racanière et al. (2017). For our purposes, a Sokoban problem is determined by four matrices W, G, B, P with sizes corresponding to the problem size. Here, $W_{xy} = 1$ if there is a *wall* at position x, y , else 0. Similarly, G determines the position of *goals*, B *boxes*, and P the *player*. The agent can perform five actions *left*, *right*, *up*, *down*, and *no-op*. Without describing the exact mechanics, the actions move the player in the specified direction, possibly pushing a box, and modify the matrices B, P accordingly. The *no-op* action does nothing. The problem is solved when all boxes are on the goal positions, i.e., $B = G$.

For each action, the agent receives a penalty -0.1 , plus a reward 1 if it pushes a box on a goal, -1 if it pushes a box off a goal, and 10 when the level is solved.

State and actions. A state is defined as a graph with a node n_{xy} for every $W_{xy} = 0$, i.e., only the playable spaces without walls. Compared to convolutional neural networks (CNN), our architecture accommodates to the particular problem and can save computational resources. The features of every n_{xy} node are defined as a concatenation of G_{xy}, B_{xy} and P_{xy} . We tried including positional x, y features, but found no difference in performance. Every two neighboring nodes (in the four directions in the grid) are connected with two opposite edges. Each edge contains a single feature determining its original direction (up, down, left, right).

Rather than using the elementary actions of the environment, we take advantage of the unique ability of our framework to define **actions that operate directly on the available objects**. We define the five following macro actions: $move-to(n)$, $push-left(n)$, $push-right(n)$, $push-up(n)$, $push-down(n)$. These actions directly select a specific node and operate upon it. All actions use a planner that maps them onto the elementary actions of the environment. If a particular action is not executable, the *no-op* action is performed. The reward for performing every action is defined as a sum of rewards resulting from the execution of the related elementary actions. The action $move-to(n)$ walks the player to a location of a node n without moving any boxes on the way, if possible (i.e., the path exists). Note that this action is not strictly necessary to solve any problem, but may facilitate some difficult situations. Other $push-(n)$ actions operate directly on the boxes at the place of node n and push them in the corresponding direction (if possible). Pre-conditions were not used, although it would be possible.

Generation. We used an implementation of Sokoban provided by Schrader (2018) and the *unfiltered* dataset from Guez et al. (2018), which contains 900k pre-generated levels of size 10x10 with 4 boxes. For randomly generated levels, we use a method described by Racanière et al. (2017).

A.3. SysAdmin

Definition. In the SysAdmin domain, an oriented graph represents a computer network. The nodes represent the computers $\mathcal{C} = \{c_1, c_2, \dots, c_N\}$ and each edge $(c_i, c_j) \in \mathcal{E}$ represents a dependency of c_j on c_i . At each timestep t , each computer can be either *online* or *offline*. Let $on_t(c) = 1$ if c is online at step t , else it is 0. Let $D(c_j) = \{c_i : (c_i, c_j) \in \mathcal{E}\}$ be a set of all computers that c_j depends on and let $D_{on_t}(c_j) = \{c_i : (c_i, c_j) \in \mathcal{E} \wedge on_t(c_i) = 1\}$ be a set of computers that c_j depends on and which are online at step t . At each step, computers which are online have a chance to shut down and offline computers have a chance to reboot and become online. Without any intervention, the network evolves as follows:

$$P(on_{t+1}(c) = 1) = \begin{cases} 0.9 \cdot \frac{1 + |D_{on_t}(c)|}{1 + |D(c)|} & \text{if } on_t(c) = 1 \\ 0.04 & \text{if } on_t(c) = 0 \end{cases}$$

At each step t , an action $reset(\mathcal{X}_t)$ can be performed. It resets the targeted computers $\mathcal{X}_t$, such that $on_{t+1}(c) = 1 : \forall c \in \mathcal{X}_t$. At each timestep, the agent receives a reward:

$$r_t = \sum_{c \in \mathcal{C}} on_t(c) - 0.75 \cdot |\mathcal{X}_t|$$

We investigate two variants of the problem: In *SysAdmin-S* (*S* for Single), only a single computer can be selected, $|\mathcal{X}_t| \leq 1$. In *SysAdmin-M* (*M* for Multi), an arbitrary set of computers can be reset.

State and actions. The graph of computers $\mathcal{C}$ and static dependencies $\mathcal{E}$ constitute the state. Each computer c has a single bit feature $on_t(c)$, determining whether it is online. The edge orientations represent the dependencies. In SysAdmin-S, two actions are available: *noop* and *reset(c)*. The first action does not select any computer to restart, second action selects one. In SysAdmin-M, there is only one action *reset(X)*, implemented as a set action, which allows to select an arbitrary set of computers.

Generation. For each node of a graph with N nodes, from 1 to 3 (uniformly chosen) other nodes become its dependees.

B. Model architecture & hyper-parameters

For all non-linear layers, we use LeakyReLU activation function (Maas et al., 2013), unless specified otherwise. Before processing the state in the GNN, features of each object are embedded into a fixed-length vector of size *emb_size*, with a shared single non-linear layer. The same parameter *emb_size* then defines the dimension of all subsequent intermediary embeddings of nodes and the global context. Edge types are one-hot-encoded and used directly. A *mp_steps* message-passing steps (eqs. 1, 2) are performed to get the final embeddings $\mathcal{V}, g$. AdamW optimizer (Loshchilov & Hutter, 2017) with a weight decay of 1×10^{-4} is used. Gradients exceeding *grad_max_norm* are normalized to this norm. Learning rate and entropy regularization coefficient α_h are annealed, from their respective starting values $LR_{start}, \alpha_{hstart}$, until their minimum LR_{end}, α_{hend} . Learning rate annealing schedule is step-based, with a factor 0.5 used every $20 \times epoch$ steps. Coefficient α_h is annealed based on $\frac{1}{t}$ schedule, where t is increased per each *epoch* steps. For each environment, we define a *q_range* interval, that is used to clip the target q in eq. 4. A batch of *p_envs* environments is simulated in parallel. Many of the parameters were found using a grid-search in their respective domains. Used resources and other hyper-parameters are available in Table A.1.

C. GNN implementation

First, let us define the input graph as follows. Let $v \in \mathcal{V}$ be a node in the set of nodes. Let $e \in \mathcal{E}$ an edge in the set of edges and let $e.s, e.r$ denote the sending and receiving nodes of this edge. Let g be a special *global* node not included in $\mathcal{V}$. For simplicity, let v, g and e also denote the feature vector of the respective node or edge.

The core of the algorithm is a single message-passing step. First, the incoming messages are aggregated:

$$\forall v : m_v = \max_{e.r=v} \phi_{msg}(e, e.s)$$

Symbolic Relational Deep Reinforcement Learning based on Graph Neural Networks

parameter	BlockWorld	Sokoban	SysAdmin-S/-M
p_envs , batch size	256	256	256
ρ , target-network update coefficient	0.005	0.005	0.005
γ , discount factor	0.99	0.99	0.99
$epoch$, number of steps per epoch	1000	1000	100
episode step-limit	100	200	100
mp_steps number of GNN message-passes	3	10	5
emb_size , embedding size	32	64	32
LR_{start} , initial learning rate	3×10^{-4}	3×10^{-3}	3×10^{-3}
LR_{end} , final learning rate	1×10^{-5}	1×10^{-4}	1×10^{-4}
$grad_max_norm$, maximal gradient	3.0	5.0	3.0
q_range , range of target q (eq. 4)	$[-15, 15]$	$[-15, 15]$	$[-100, 200 \cdot N]$
α_v , coefficient of L_V	0.1	0.1	0.1
α_{hstart} , initial coefficient of L_H	1×10^{-4}	0.2	0.3/0.3/1.0/1.0/2.0/2.0 in -S 0.3/0.3/3.0/10/20/24 in -M for $N = 5/10/20/40/80/160$
α_{hend} , final coefficient of L_H	5×10^{-5}	0.1	$\frac{1}{2}\alpha_{hstart}$
resources used in training	4 CPU cores	2 CPU cores, 1 GPU	1 CPU core

Table A.1: Hyper-parameters and other settings used in the experiments

Here, ϕ_{msg} is a message embedding function that transforms an incoming message from node $e.s$ over an edge e . The results are aggregated with an element-wise max function. Another common aggregation operator is $mean$ (Battaglia et al., 2018); we choose max early in our experiments, where it worked best. Second, all node features are updated with newly computed values:

$$\forall v : v' = v + \phi_{agg}(v, m_v, g) \quad (1)$$

The messages m_v are processed with function ϕ_{agg} , which also takes the current embedding of v and the global node features g . In practice, we implement the ϕ_{msg} and ϕ_{agg} functions as single non-linear neural network layers. The addition of the original v represents a skip connection (Kipf & Welling, 2017; He et al., 2016), which we found to facilitate learning if the number of message-passing steps is large. After all node representations are updated, a global node g aggregates information from all other nodes through an attention mechanism:

$$g' = g + \phi_{glb}\left(g, \sum_{v \in \mathcal{V}} \phi_{att}(v) \cdot \phi_{feat}(v)\right) \quad (2)$$

The ϕ_{att} denotes a softmax distribution over all nodes in $\mathcal{V}$, ϕ_{feat} a node embedding function and ϕ_{glb} is a final embedding function. In the implementation, ϕ_{att} is a single linear layer followed by softmax and ϕ_{feat} and ϕ_{glb} are single non-linear layers. Again, adding the original g serves as a skip connection to facilitate learning.

The steps (1) and (2) form a single message-passing step. Several steps result in final embeddings $v \in \mathcal{V}$ and g . We use independent parameters for the ϕ functions for each step (see Battaglia et al., 2018). In this way, the model can compute progressively more complex representations.

D. A2C algorithm

Below we present our implementation of the A2C algorithm (Mnih et al., 2016) with a target network (Lillicrap et al., 2016) and entropy gradient sampling (Zhang et al., 2018).

Let π_θ be a policy and V_θ a value estimate, where θ are model parameters. In a Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, r, t, \gamma)$, where $\mathcal{S}, \mathcal{A}$ are state and action spaces, r, t are reward and transition functions and γ is a discount factor, let $Q(s, a) = \mathbb{E}_{s' \sim t(s, a)}[r(s, a, s') + \gamma V_\theta(s')]$ be a state-action value function and $A(s, a) = Q(s, a) - V_\theta(s)$ an advantage function. Then, the policy gradient $\nabla_\theta J$ and the value function loss L_V are:

$$\nabla_\theta J = \mathbb{E}_{s, a \sim \pi_\theta, t} \left[A(s, a) \cdot \nabla_\theta \log \pi_\theta(a|s) \right] \quad (3)$$

$$L_V = \mathbb{E}_{s,a,s' \sim \pi_{\theta}, t} \left[q(s, a, s') - V_{\theta}(s) \right]^2 \quad (4)$$

where the target q is:

$$q(s, a, s') = \begin{cases} r(s, a, s') & \text{if } s' \text{ is terminal} \\ r(s, a, s') + \gamma V_{\theta'}(s') & \text{else} \end{cases}$$

To prevent a target run-away problem in L_V , the $V_{\theta'}(s')$ is estimated using a copy of parameters θ' that are regularly updated with $\theta' := (1 - \rho)\theta' + \rho\theta$, with $\rho \in (0, 1]$. An entropy regularization term L_H is:

$$L_H = \mathbb{E}_{s \sim \pi_{\theta}, t} \left[H_{\pi_{\theta}}(s) \right]; \quad H_{\pi}(s) = - \mathbb{E}_{a \sim \pi(s)} \left[\log \pi(a|s) \right] \quad (5)$$

However, the precise computation of the policy entropy is intractable in our case – only single $\pi(a|s)$ for the actually performed action a is available. Therefore, we rewrite the entropy in its gradient form and sample the expectation (Zhang et al., 2018):

$$\nabla_{\theta} H_{\pi_{\theta}}(s) = - \mathbb{E}_{a \sim \pi_{\theta}(s)} \left[\log \pi_{\theta}(a|s) \cdot \nabla_{\theta} \log \pi_{\theta}(a|s) \right]$$

The final gradient is $\nabla_{\theta}(-J + \alpha_v L_V - \alpha_h L_H)$, with α_v, α_h being learning rate coefficients. We simulate a batch of parallel environments to gather a better gradient estimate. Per each step of the environment, we perform a single gradient step.

E. Other details

We measured the number of optimal steps in BlockWorld using Fast Downward planner (Helmert, 2006) with A* algorithm and LM-cut heuristic (Helmert & Domshlak, 2011). The PROST planner used in SysAdmin domain is evaluated with the preset of International Probabilistic Planning Competition 2014.

E.1. Normalization of the entropy regularization

In the implementation of A2C, we normalize the entropy regularization term with a maximal entropy of a particular state. The intuition is that because the number of actions is state-dependent, the maximal magnitude of the L_H term varies. However, our tests are yet inconclusive whether this technique brings any benefit; hence we describe it here as a pure implementation detail. Let $|\mathcal{A}(s)|$ be the number of actions in a particular state s , then the maximal policy entropy is $H_{max}(s) = \log |\mathcal{A}(s)|$; the proof is trivial. To stabilize the training, we propose to normalize the L_H term with the maximal possible entropy in a particular state: $L_H = \mathbb{E}_{s \sim \pi_{\theta}, t} \left[\frac{H_{\pi_{\theta}}(s)}{H_{max}(s)} \right]$. In states with only a single available action, $H_{max}(s)$ is zero and such state is excluded from the L_H computation.